

Overflow and scrolling

In the last lesson, you positioned a few elements outside normal flow.

This lesson looks at a different kind of problem: content that is larger than the box it lives inside.

That happens all the time in real interfaces:

- A notification panel has more messages than fit in the space.
- A code sample is wider than the card around it.
- A long title does not fit on one row.
- An image or decorative shape needs to be clipped to a rounded container.

Overflow is the browser's answer to one question:

What should happen when the content is bigger than the box?

Start with content that outgrows a box

Use a small release-notes page. It has a normal article, a messages panel, a long code sample, and one long lesson title.

The page is intentionally built so a few pieces have too much content for their space.

Start with this HTML:

html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>Overflow practice</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <main class="page">
      <section class="intro">
        <p class="eyebrow">CSS practice</p>
        <h1>Decide what happens when content does not fit</h1>
        <p class="summary">Some content should make the page taller. Some content should
      </section>

      <section class="layout">
        <article class="release-card">
          <h2>Release notes</h2>
          <p>Version 2.4 improves layout tools, fixes several mobile spacing issues, and
```



```
<pre class="code-sample"><code>pnpm roadmap lesson publish --pack css --lesson 1</code></pre>

<h3 class="long-title">A very long lesson title that needs to stay on one line</h3>
<p>The title above is not a paragraph. It is a compact label, so it may need a compact treatment.</p>
</article>

<aside class="message-panel">
  <h2>Recent messages</h2>

  <div class="message-list">
    <p><strong>Ada:</strong> The card layout looks good on desktop.</p>
    <p><strong>Sam:</strong> The mobile header needs another pass.</p>
    <p><strong>Priya:</strong> The code sample is making the page too wide.</p>
    <p><strong>Lee:</strong> The release notes panel should not take over the whole page.</p>
    <p><strong>Noor:</strong> The long title needs a compact treatment.</p>
    <p><strong>Maya:</strong> The final screenshot is ready for review.</p>
  </div>
</aside>
</section>
```

Then add the base CSS:

CSS

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #111827;
  background-color: #f3f4f6;
}

.page {
  max-width: 1040px;
  margin: 40px auto;
  padding: 0 16px;
}

.intro,
.release-card,
.message-panel {
  background-color: white;
  border: 1px solid #e5e7eb;
  border-radius: 8px;
}

.intro,
.release-card,
.message-panel {
  padding: 24px;
}

.intro {
  margin-bottom: 24px;
}

.eyebrow {
  margin-top: 0;
  color: #2563eb;
  font-size: 14px;
  font-weight: 700;
}

h1,
h2,
h3,
p {
  margin-top: 0;
}

```

```
}

.summary,
.release-card p,
.message-list p {
  color: #4b5563;
  line-height: 1.6;
}

.layout {
  display: flex;
  gap: 24px;
  align-items: flex-start;
}

.release-card {
  flex: 1;
}

.message-panel {
  width: 320px;
}

.code-sample {
  padding: 16px;
  color: #e5e7eb;
  background-color: #111827;
  border-radius: 8px;
}

.long-title {
  padding: 12px;
  background-color: #eff6ff;
  border-radius: 8px;
}
```

Open the page and look for two problems:

- The code sample may push sideways because the command is one long line.
- The messages panel can become taller than you want because it has many messages.

That is the setup: overflow only becomes a real question when a box has less space than its content wants.

Overflow needs a constrained box

Overflow problems happen when you have more content than a constrained box can contain. So far, most of the pages in this pack have simply grown taller when they needed more vertical space.

Before we discuss **overflow**, the browser needs a box with a limit.

If a normal section has a lot of text, the browser usually makes the section taller. No overflow problem appears because the box grows to fit the content.

To create a real overflow decision, give the messages list a maximum height:

CSS	

```
.message-list {  
  max-height: 220px;  
  padding-right: 8px;  
}
```

max-height: 220px says the list may be shorter, but it should not grow taller than **220px** .

Now the messages may not all fit inside that height. The content is larger than the box.

padding-right: 8px adds a little breathing room on the right. It will matter once a scrollbar appears.

At this point, the browser still uses the default overflow behavior.

The default is visible overflow

The default value is **overflow: visible** , which means content is allowed to paint outside the box.

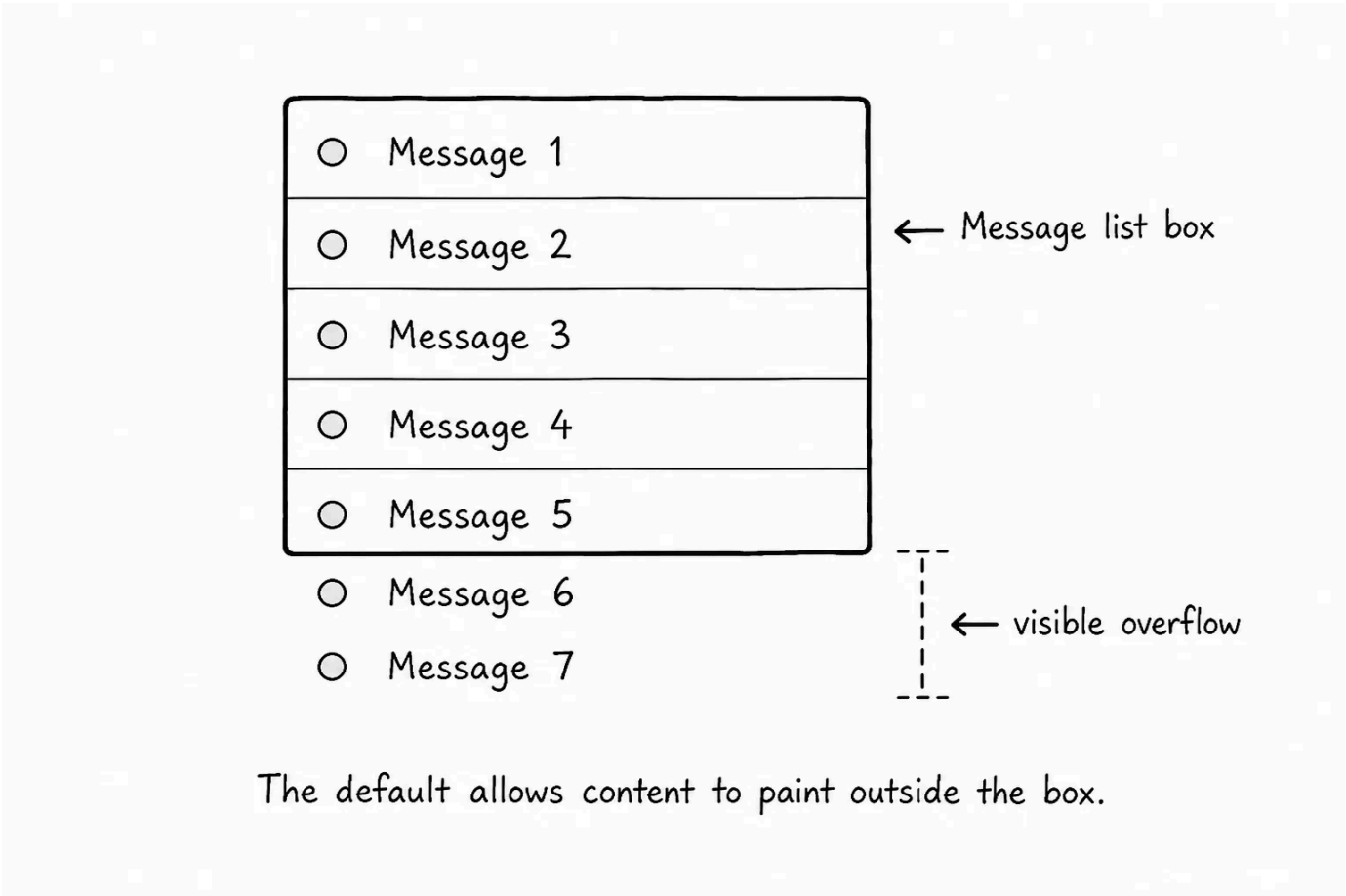
Add it explicitly so you can see the default:

CSS

```
.message-list {  
  max-height: 220px;  
  padding-right: 8px;  
  overflow: visible;  
}
```

The list has a height limit, but the extra messages may still spill out visually.

That can be useful for some decorative effects, but it is not good for a message panel. The panel now says it has one size while its content paints outside that space.



Use auto when the user should scroll

For a message list, the extra content is still important. You should not hide it because the user needs a way to reach it.

That is a good job for `overflow: auto`.

Change the list to use automatic scrolling:

CSS

```
.message-list {
  max-height: 220px;
  padding-right: 8px;
  overflow: auto;
}
```

`overflow: auto` lets the browser add scrolling only when the content needs it.

If the messages fit, there may be no scrollbar. If the messages do not fit, the list becomes scrollable.

That is why `auto` is usually better than forcing scrollbars everywhere: it lets the content decide.

Use this for panels, menus, sidebars, log viewers, notification lists, and other bounded areas where the user should still reach all the content.

 **Prefer auto for real content** - If the content matters and the box has a size limit, start with `overflow: auto`. It preserves access to the content while keeping the layout under control.

Use hidden only when clipping is intentional

Sometimes overflow should be clipped. For example, a decorative image inside a rounded card might need to stay inside the rounded corners, a badge glow might need to stay inside a compact component, or a preview thumbnail might crop extra visual detail.

That is when `overflow: hidden` can make sense.

Add a small clipped preview to the card:

html

```
<div class="preview-box">
  <div class="preview-shape"></div>
</div>
```

Place it inside `.release-card`, above the code sample, then add this CSS:

CSS

```
.preview-box {
  height: 120px;
  margin-bottom: 16px;
  background-color: #dbeafe;
  border-radius: 12px;
  overflow: hidden;
}

.preview-shape {
  width: 180px;
  height: 180px;
  margin-left: auto;
  background-color: #60a5fa;
  border-radius: 999px;
}
```

The circle is taller than the preview box. `overflow: hidden` clips the part that sticks out.

This works because the extra part is decorative. The user is not losing an important message or control.

That distinction matters because `overflow: hidden` does not delete content from the HTML. It only makes the overflowing part invisible and unreachable visually.

 **Do not hide important content** - Avoid using `overflow: hidden` to make real text, form fields, buttons, or warnings disappear. If users need the content, use a layout that fits it or use scrolling.

Control one direction when needed

Sometimes only one direction is the problem.

The code sample is a good example. The command is one long line. You do not want the whole page to become wider just because one command is wide.

The fix is horizontal scrolling on the code box:

CSS

```
.code-sample {
  padding: 16px;
  color: #e5e7eb;
  background-color: #111827;
  border-radius: 8px;
  overflow-x: auto;
}
```

`overflow-x` controls horizontal overflow, and `auto` means the browser only adds horizontal scrolling when the content is wider than the box.

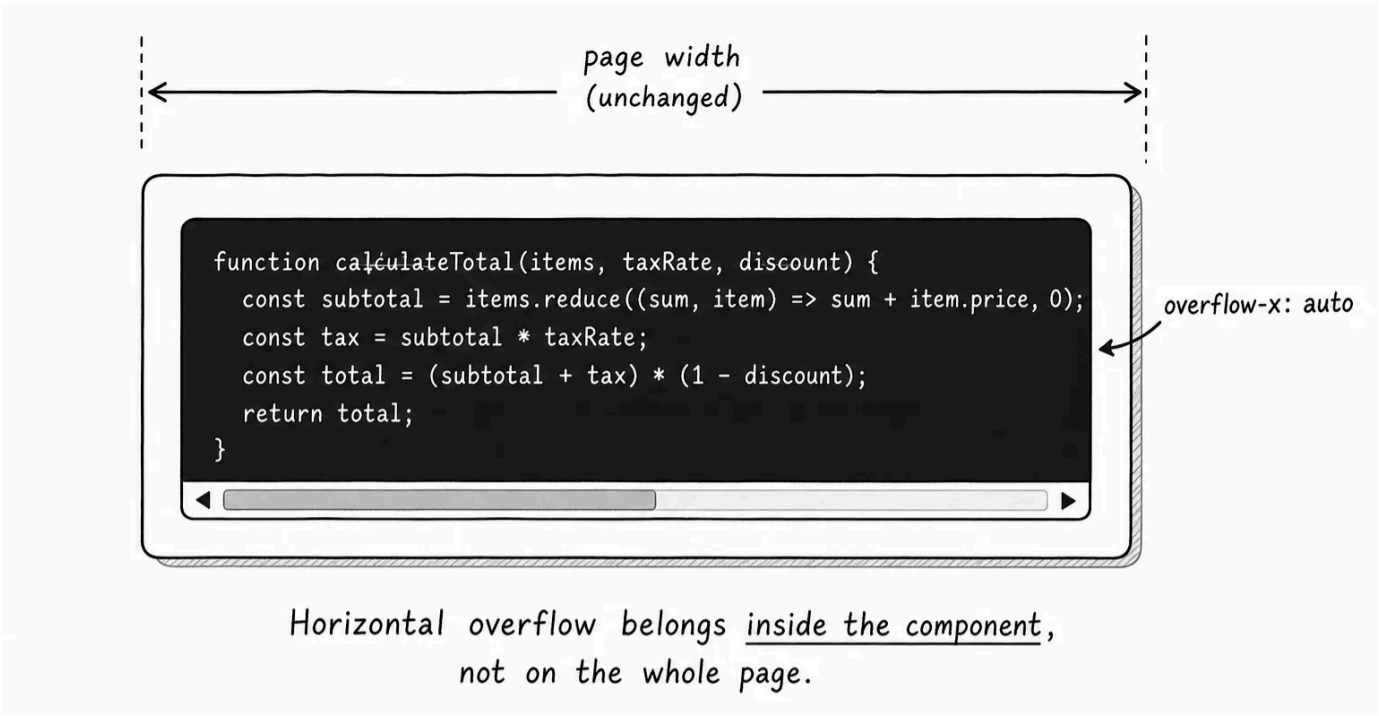
There is also **overflow-y** for vertical overflow. The messages list could be written like this:

CSS

```
.message-list {
  max-height: 220px;
  padding-right: 8px;
  overflow-y: auto;
}
```

overflow-y: auto focuses the scroll behavior on the vertical direction.

Use directional overflow when the problem is clearly one direction: wide tables, long code lines, tag rows, or tall panels.



Keep the page from getting wider by accident

Horizontal page overflow is one of the most common CSS bugs.

You see it when the whole page gets a sideways scrollbar. Usually, one child element is wider than the viewport or wider than its parent.

The release card is a flex item because **.layout** uses Flexbox. The card is supposed to take the available space:

CSS

```
.release-card {
  flex: 1;
}
```

flex: 1 lets the card take the available space in the row.

But Flexbox has a detail that can surprise you: flex items have an automatic minimum size based on their content. A long code line can make the card refuse to shrink.

Add this:

CSS

```
.release-card {  
  flex: 1;  
  min-width: 0;  
}
```

min-width: 0 allows the card to shrink inside the layout, so the code block can handle its own horizontal scrolling.

This is not an overflow property, but it often fixes overflow bugs in Flexbox layouts.

If a page has a mystery horizontal scrollbar, inspect the wide child first. Then check whether a flex item needs **min-width: 0**.

Truncate one-line text carefully

The long title is not normal paragraph text. It is a compact card heading. In some interfaces, you may want it to stay on one line and show that it was shortened.

That needs three declarations working together:

CSS

```
.long-title {  
  padding: 12px;  
  background-color: #eff6ff;  
  border-radius: 8px;  
  white-space: nowrap;  
  overflow: hidden;  
  text-overflow: ellipsis;  
}
```

white-space: nowrap keeps the title on one line.

overflow: hidden clips the part that does not fit.

text-overflow: ellipsis shows three dots at the clipped edge, so the user can see that the text continues.

The ellipsis only works when the text is forced onto one line and the overflow is hidden.

Use this carefully. It is fine for compact labels, table cells, filenames, and cards where the full text is available somewhere else. It is a poor choice for important instructions or body text.

Inspect overflow in DevTools

DevTools helps you find overflow instead of guessing.

Inspect the code sample and look at its size. Toggle `overflow-x: auto` off. If the page gets wider or the code spills out, you have found the reason for the horizontal overflow.

Inspect the messages list and toggle `overflow: auto` off. The extra messages should spill outside the constrained box again.

If you see a horizontal scrollbar on the whole page, use the element picker and hover wide elements. Look for something that is wider than its parent or wider than the viewport.

Overflow bugs usually come from one of these:

- A fixed width that is too large for the screen.
- A long unbroken line of text or code.
- A flex item that needs `min-width: 0`.
- An image or media element that is not allowed to shrink.
- A positioned or fixed element that reaches past the viewport edge.

The fix depends on the cause. Do not reach for `overflow: hidden` on the `body` as the first move. That can hide the symptom while leaving the broken layout in place.

Overflow values in one place

Here is the practical map:

Value	What it does
<code>visible</code>	Lets content paint outside the box. This is the default.
<code>auto</code>	Adds scrolling only when the content needs it.
<code>scroll</code>	Shows scrollbars even when they may not be needed.
<code>hidden</code>	Clips overflowing content and does not show scrollbars.

You can apply the idea to both directions with `overflow-x` and `overflow-y`.

The useful question is: should the user still be able to reach the content?

If yes, use scrolling or let the layout grow. If no, clipping may be acceptable.

Try it

Use the same page and test each overflow decision:

- Change `.message-list` from `overflow: auto` to `overflow: visible` . Notice how the messages spill outside the constrained height.
- Change `.message-list` to `overflow: hidden` . Notice that the extra messages are clipped and no scrollbar appears.
- Add two more messages inside `.message-list` . Check whether `overflow: auto` adds scrolling only when needed.
- Remove `overflow-x: auto` from `.code-sample` . Watch whether the code line pushes outside its box or makes the page wider.
- Remove `min-width: 0` from `.release-card` . If the page gets wider, inspect the card and code sample to see what is refusing to shrink.
- Remove `white-space: nowrap` from `.long-title` . Notice why the ellipsis no longer behaves the same way.
- Use DevTools to toggle each overflow declaration and watch what changes before saving the final version.

The habit is simple: first decide whether the content should stay reachable. Then choose growing, scrolling, clipping, or truncating based on that decision.

< 0 / 10 >

✔ Knew 0 Items

🌟 Learnt 0 Items

⏮ Skipped 0 Items

🔄 ResetProgress

Test Your Understanding

What is overflow?

[Click to Reveal the Answer](#)

✔ Already Know that

🌟 Didn't Know that

⏮ Skip Question

✓

LESSON COMPLETE

Nicely done.

Up next: Personal Portfolio

Next Lesson →